FRONTEND SOURCE CODE: BOOKING PAGE

src/routes/book.tsx

```
function emptyBookingForm() {
  return {
    firstName: "",
    middleName: "",
    lastName: "",
    phone: "",
    email: "",
    motoBrand: "",
    motoModel: "",
    motoVariant: "",
    motoYear: String(
      new Date().getFullYear(),
    ),
    plateNumber: "",
    notes: "",
    wantsProducts: null as
      boolean | null,
  };
}

// ...

const BOOKING_PAGES = [
  "Booking details",
  "Review & submit",
] as const;

export const Route =
  createFileRoute("/book")({
    validateSearch: searchSchema,
    head: () => ({
      meta: [
        {
          title:
            "Book a Service Appointment |
              Fake Rider Motorparts",
        },
        {
          name: "description",
          content:
            "Reserve your motorcycle
              service slot online. Choose
              your services and schedule a
              convenient time, then save
              your reference code.",
        },
        {
          property: "og:title",
          content:
            "Book a Service Appointment |
              Fake Rider",
        },
        {
          property:
            "og:description",
          content:
            "Reserve your motorcycle
              service slot online in a few
              minutes.",
        },
      ],
    }),
    component: BookPage,
  });

type Errors = Partial<{
  firstName: string;
  middleName: string;
  lastName: string;
  phone: string;
  email: string;
  motoBrand: string;
  motoModel: string;
  plateNumber: string;
  services: string;
  wantsProducts: string;
  date: string;
  startTime: string;
  schedule: string;
  terms: string;
  rescheduleReason: string;
}>;

// ...
```

```
const availability =
  useQuery<Availability>({
    queryKey: [
      "availability",
      serviceIds,
      form.motoBrand,
      form.motoModel,
      isReschedule,
      serviceIds.length > 1,
    ],
    queryFn: () =>
      availabilityFn({
        data: {
          days: 45,
          serviceIds,
          rescheduling:
            isReschedule,
          allowMultiDayContinuation:
            !isReschedule &&
            serviceIds.length > 1,
          motorcycle: {
            brand: form.motoBrand,
            model: form.motoModel,
          },
          ...(search.reschedule &&
          search.phone
            ? {
                rescheduleReference:
                  search.reschedule,
                reschedulePhone:
                  normalizePhilippineMobile(
                    search.phone,
                  )!,
              }
            : {}),
        },
      }),
    enabled:
      serviceIds.length > 0 &&
      motorcycleSelectionReady,
  });

const dates = useMemo(
  () =>
    availability.data
      ? computeAvailableDates(
          availability.data,
        )
      : [],
  [availability.data],
);

const slots = useMemo(
  () =>
    availability.data && date
      ? computeAvailableSlots(
          availability.data,
          date,
        )
      : [],
  [availability.data, date],
);

// ...

const mutation = useMutation({
  mutationFn: async () => {
    if (
      form.wantsProducts === null
    ) {
      throw new Error(
        "Choose whether you would like
          to avail shop products.",
      );
    }
    const submittedSchedule = {
      date,
      startTime,
    };
    const res = await book({
      data: {
        firstName:
          form.firstName.trim(),
        middleName:
          form.middleName.trim(),
        lastName:
          form.lastName.trim(),
        phone:
          normalizePhilippineMobile(
            form.phone,
          )!,
        email: form.email
```

```
        .trim()
        .toLowerCase(),
      motoBrand:
        form.motoBrand.trim(),
      motoModel:
        form.motoModel.trim(),
      motoVariant:
        form.motoVariant.trim(),
      motoYear: Number(
        form.motoYear,
      ),
      plateNumber:
        form.plateNumber.trim(),
      serviceIds,
      date,
      startTime,
      notes: form.notes.trim(),
      wantsProducts:
        form.wantsProducts,
      turnstileToken,
      idempotencyKey:
        bookingRequestId,
      termsAccepted:
        true as const,
      multiDayContinuationAccepted,
      ...(isReschedule
        ? {
            rescheduleReference:
              search.reschedule,
          }
        : {}),
      ...(isReschedule
        ? {
            rescheduleReason:
              rescheduleReason.trim(),
          }
        : {}),
    },
  });
  return {
    res,
    submittedSchedule,
  };
},
onSuccess: ({
  res,
  submittedSchedule,
}) => {
  if (!res.ok) {
    if (
      !isReschedule &&
      isDuplicateBookingMessage(
        res.error,
      )
    ) {
      setDuplicateBookingMessage(
        res.error,
      );
      setErrors(
        ({
          schedule: _,
          ...current
        }) => current,
      );
      setDate("");
      setStartTime("");
      setTurnstileToken("");
      setBookingRequestId(
        randomId(),
      );
      availability.refetch();
      return;
    }
    setErrors((current) => ({
      ...current,
      schedule: res.error,
    }));
    setTurnstileToken("");
    setBookingRequestId(
      randomId(),
    );
    availability.refetch();
    return;
  }
  clearBookingDraft();
  setResult(
    "continuationSegments" in res
      ? {
          reference:
            res.reference,
          total: res.total,
          date: submittedSchedule.date,
```

```
          startTime:
            submittedSchedule.startTime,
          continuationSegments:
            res.continuationSegments,
        }
      : {
          reference:
            res.reference,
          total: res.total,
          date: submittedSchedule.date,
          startTime:
            submittedSchedule.startTime,
        },
  );
  window.scrollTo({
    top: 0,
    behavior: "smooth",
  });
},
onError: (err: Error) => {
  if (
    !isReschedule &&
    isDuplicateBookingMessage(
      err.message,
    )
  ) {
    setDuplicateBookingMessage(
      err.message,
    );
    setErrors(
      ({
        schedule: _,
        ...current
      }) => current,
    );
    setDate("");
    setStartTime("");
    return;
  }
  const msg =
    err.message.toLowerCase();
  let scheduleError: string;
  if (
    msg.includes("network") ||
    msg.includes("fetch") ||
    msg.includes("connect")
  ) {
    scheduleError =
      "Network error. Please check
        your connection and try again.";
  } else if (
    msg.includes("timeout")
  ) {
    scheduleError =
      "The request timed out. Please
        try again.";
  } else if (
    msg.includes("validation") ||
    msg.includes("invalid")
  ) {
    scheduleError =
      "Some fields have invalid values.
        Please review the form.";
  } else {
    scheduleError =
      "We could not submit your
        booking. Please review your
        details and try again.";
  }
  setErrors((current) => ({
    ...current,
    schedule: scheduleError,
  }));
  availability.refetch();
 },
});

function validationErrors(
  steps: number[],
) {
  const e: Errors = {};
  if (
    isReschedule &&
    !rescheduleReady
  ) {
    e.schedule =
      rescheduleError ??
      "Loading the original appointment
        details.";
    return e;
  }
  if (steps.includes(1)) {
```

```
    if (
      form.lastName.trim().length <
      2
    )
      e.lastName =
        "Enter your last name.";
    if (
      form.firstName.trim().length <
      2
    )
      e.firstName =
        "Enter your first name.";
    if (
      !normalizePhilippineMobile(
        form.phone,
      )
    )
      e.phone =
        PHONE_VALIDATION_MESSAG
          E;
    if (
      !isReschedule &&
      !/^\S+@\S+\.\S+$/.test(
        form.email.trim(),
      )
    ) {
      e.email =
        "Enter a valid email address.";
    }
  }
  if (steps.includes(2)) {
    if (!form.motoBrand.trim())
      e.motoBrand = "Required";
    if (!form.motoModel.trim())
      e.motoModel = "Required";
    if (
      !isReschedule &&
      catalogUnavailable
    ) {
      e.motoBrand =
        "The motorcycle catalog is
          unavailable. Please try again
          shortly.";
    }
    if (
      !isReschedule &&
      !catalogUnavailable &&
      form.motoBrand &&
      !brands.includes(
        form.motoBrand,
      )
    ) {
      e.motoBrand =
        "Select a brand from the
          motorcycle catalog.";
    }
    if (
      !isReschedule &&
      !catalogUnavailable &&
      form.motoModel &&
      !modelsForBrand.some(
        (model) =>
          model.value ===
          form.motoModel,
      )
    ) {
      e.motoModel =
        "Select a model from the
          motorcycle catalog.";
    }
    if (
      form.plateNumber.trim()
        .length < 2
    )
      e.plateNumber = "Required";
  }
  if (
    steps.includes(3) &&
    serviceIds.length === 0
  )
    e.services =
      "Select at least one service.";
  if (
    steps.includes(3) &&
    serviceIds.length >
      MAX_BOOKING_SERVICE_SELE
        CTIONS
  ) {
    e.services = `Select up to
      ${MAX_BOOKING_SERVICE_SE
      LECTIONS} services per
      appointment.`;
```

```
}
if (steps.includes(4) && !date)
  e.date =
    "Select your preferred date.";
if (
  steps.includes(4) &&
  date &&
  !startTime
)
  e.startTime =
    "Select your preferred time.";
if (
  steps.includes(3) &&
  form.wantsProducts === null
) {
  e.wantsProducts =
    "Choose Yes or No for shop
      products.";
}
if (
  steps.includes(4) &&
  availabilityLoading
) {
  e.schedule =
    "Availability is still loading. Please
      wait a moment.";
}
if (
  steps.includes(4) &&
  (availability.isError ||
    availabilityError)
) {
  e.schedule =
    availabilityError ??
    "We could not verify availability.
      Please try again.";
}
if (
  steps.includes(4) &&
  isReschedule &&
  !rescheduleReason.trim()
) {
  e.rescheduleReason =
    "Tell us why you need to
      reschedule.";
}
  if (steps.includes(5) && !terms)
    e.terms =
      "You must accept the terms and
        conditions.";
  return e;
}

function validateBooking() {
  const e = validationErrors([
    1, 2, 3, 4, 5,
  ]);
  return showValidationErrors(e);
}

function showValidationErrors(
  e: Errors,
) {
  setErrors(e);
  const firstInvalidField =
    validationFieldOrder.find(
      (field) => e[field],
    );
  if (!firstInvalidField)
    return true;

  setBookingPage(
    firstInvalidField === "terms"
      ? "review"
      : "details",
  );
  setFocusRequest(
    firstInvalidField,
  );
  return false;
}

function continueToReview() {
  const e = validationErrors([
    1, 2, 3, 4,
  ]);
  if (!showValidationErrors(e))
    return;
  const requiresMultiDayConfirmation
    =
    needsServiceStepMultiDayConfirm
      ation ||
```

```
    needsMultiDayContinuationConfirm
      ation;
  if (
    requiresMultiDayConfirmation &&
    !multiDayContinuationAccepted
  ) {
    setMultiDayContinuationOpen(
      true,
    );
    return;
  }
  setBookingPage("review");
  window.scrollTo({
    top: 0,
    behavior: "smooth",
  });
}

function
  continueWithMultiDayBooking() {
  setMultiDayContinuationAccepted(
    true,
  );
  setMultiDayContinuationOpen(
    false,
  );
  setBookingPage("review");
  window.scrollTo({
    top: 0,
    behavior: "smooth",
  });
}

function returnToBookingDetails() {
  setErrors({});
  setBookingPage("details");
  window.scrollTo({
    top: 0,
    behavior: "smooth",
  });
}

function submitBooking() {
  if (
    !rescheduleReady ||
    !validateBooking()
  )
    return;
  mutation.mutate();
}

// ...

<Section
  step={1}
  title="Personal information"
  description={
    isReschedule
      ? "These details come from your
        original appointment and are
        verified when you submit."
      : "We use these details to confirm
        your booking and contact you
        about it."
  }
  complete={
    sectionComplete.personal
  }
>
  <div className="grid gap-4
    sm:grid-cols-2 lg:grid-cols-3">
    <Field
      id="booking-last-name"
      label="Last name"
      error={errors.lastName}
    >
      <Input
        id="booking-last-name"
        value={form.lastName}
        maxLength={40}
        autoComplete="family-name"
        disabled={isReschedule}
        aria-invalid={Boolean(
          errors.lastName,
        )}
        onChange={(e) =>
          updateForm(
            "lastName",
            formatNamePartInput(
              e.target.value,
            ),
          )
```

```
    }
    placeholder="Dela Cruz"
  />
</Field>
<Field
  id="booking-first-name"
  label="First name"
  error={errors.firstName}
>
  <Input
    id="booking-first-name"
    value={form.firstName}
    maxLength={40}
    autoComplete="given-name"
    disabled={isReschedule}
    aria-invalid={Boolean(
      errors.firstName,
    )}
    onChange={(e) =>
      updateForm(
        "firstName",
        formatNamePartInput(
          e.target.value,
        ),
      )
    }
    placeholder="Juan"
  />
</Field>
<Field
  id="booking-middle-name"
  label="Middle name (optional)"
  error={errors.middleName}
>
  <Input
    id="booking-middle-name"
    value={form.middleName}
    maxLength={40}
    autoComplete="additional-name"
    disabled={isReschedule}
    aria-invalid={Boolean(
      errors.middleName,
    )}
    onChange={(e) =>
      updateForm(
        "middleName",
        formatNamePartInput(
          e.target.value,
        ),
      )
    }
    placeholder="Santos"
  />
</Field>
<Field
  id="booking-phone"
  label="Mobile number"
  hint="11 digits, starting with 09."
  error={errors.phone}
>
  <Input
    id="booking-phone"
    type="tel"
    value={form.phone}
    maxLength={11}
    inputMode="tel"
    autoComplete="tel"
    disabled={isReschedule}
    aria-invalid={Boolean(
      errors.phone,
    )}
    aria-describedby="booking-phon
      e-hint"
    onChange={(e) =>
      updateForm(
        "phone",
        sanitizePhilippineMobileInput(
          e.target.value,
        ),
      )
    }
    placeholder="09171234567"
  />
</Field>
<Field
  id="booking-email"
  label="Email address"
  hint="We'll email your confirmed
    appointment details here."
  error={errors.email}
>
  <Input
```

```
        id="booking-email"
        type="email"
        value={form.email}
        maxLength={254}
        autoComplete="email"
        disabled={isReschedule}
        aria-invalid={Boolean(
          errors.email,
        )}
        aria-describedby="booking-email-
          hint"
        onChange={(e) =>
          updateForm(
            "email",
            e.target.value.toLowerCase(),
          )
        }
        placeholder="juan@example.co
          m"
      />
    </Field>
  </div>
</Section>

// ...

function Section({
  id,
  step,
  title,
  description,
  complete,
  error,
  className,
  children,
}: {
  id?: string;
  step?: number;
  title: string;
  description?: string;
  complete?: boolean;
  error?: string | undefined;
  className?: string;
  children: React.ReactNode;
}) {
  const titleId = useId();

return (
  <section
    id={id}
    tabIndex={id ? -1 : undefined}
    aria-labelledby={titleId}
    className={cn(
      "rounded-xl border
        border-border/70 bg-card/40 p-4
        focus:outline-none sm:p-6",
      className,
    )}
  >
    <div className="mb-5 flex
      items-start gap-3">
      {step !== undefined && (
        <span
          aria-hidden="true"
          className={cn(
            "flex size-8 shrink-0
              items-center justify-center
              rounded-full font-display
              text-base transition-colors",
            complete
              ? "bg-primary
                text-primary-foreground"
              : "bg-primary/10 text-primary
                ring-1 ring-primary/30",
          )}
        >
          {complete ? (
            <Check className="size-4"
              />
          ) : (
            step
          )}
        </span>
      )}
      <div className="min-w-0">
        <h2
          id={titleId}
          className="font-display
            text-xl tracking-wide
            uppercase"
        >
          {title}
          {complete && (
```

```
            <span className="sr-only">
              {" "}
              (complete)
            </span>
          )}
        </h2>
        {description && (
          <p className="mt-0.5 text-sm
            text-muted-foreground">
            {description}
          </p>
        )}
      </div>
    </div>
    {children}
    <FieldError
      message={error}
      className="mt-3"
    />
  </section>
 );
}

function Field({
 id,
 label,
 hint,
 error,
 children,
}: {
 id: string;
 label: string;
 hint?: string;
 error?: string | undefined;
 children: React.ReactNode;
}) {
 return (
  <div
    data-invalid={!!error}
    className="space-y-1.5
      data-[invalid=true]:[&_[role=comb
      obox]]:border-destructive
      data-[invalid=true]:[&_input]:bord
      er-destructive
      data-[invalid=true]:[&_textarea]:b
      order-destructive"
  >
    <Label htmlFor={id}>
     {label}
    </Label>
    {children}
    {error ? (
     <FieldError
       message={error}
     />
    ) : hint ? (
     <p
       id={`${id}-hint`}
       className="text-xs
         text-muted-foreground"
     >
       {hint}
     </p>
    ) : null}
  </div>
 );
}
```